Your team's study guide

CSC 212 | Keep the original A / B / C split

Start here: the finished application is one project, but each person has a clear area to explain. Learn the flow first, then study your assigned classes. Understand the examples well enough to change a value and predict what happens.

Person	Main focus	Start with
Osamah - A 443102410	How records are stored, ordered, searched and removed.	LinkedList.java RiderList.java DriverList.java RideList.java
Ibrahim - B 445100804	How the system accepts/rejects rides, loads files and cleans related records.	RideSharingSystem.java
Mohammed - C 445100816	What objects represent, inheritance, the menu, and report assembly.	Person/Rider/Driver DateTime and ride models Main.java and Report.pdf

One sentence everyone should understand

Main asks the user, RideSharingSystem checks the rules, the model classes represent the records, and the custom lists store them.

A successful private ride, from beginning to end

1. Main reads pickup, times, destination, driver ID and rider ID.
2. RideSharingSystem checks valid input and finds both people.
3. It scans both histories for a time overlap.
4. It creates one PrivateRide object and adds it to RideList.
5. It adds references to that same ride to the rider and driver histories.
6. Main prints whether the operation succeeded.

How to practice

Compile and run run.bat --demo. Open the classes named in your row while following the walkthroughs below. Use test.bat to check the full project. A test failure identifies behavior to inspect; it is not a replacement for understanding the code.

A - Osamah: the data structures

What is a linked list?

Each Node stores data and a next reference. head points to the first node; tail points to the last; the last next is null. size records the number of nodes. A list stores references to records, not a separate full copy of each record.

Example: head -> rider 10 -> rider 30 -> null. To insert rider 20, connect 10.next to the new 20 node, then 20.next to 30. The code finds the insertion index first and LinkedList.add(index,item) changes the links.

The cursor is your own traversal helper

cursor.hasNext() asks whether another node exists. cursor.next() returns that node's data and advances. cursor.remove() unlinks the most recently returned node. It remembers the previous node, so removal during a scan is O(1). It also handles removing head or tail.

Why two lists for people?

The assignment asks for storage sorted by ID and alphabetical listing in O(n). Each person list keeps an ID-ordered list and a name-ordered index. Both point to the same people. Adding/removing updates both. getAll() copies the name-ordered list in one pass.

What to explain in the code

Find this method	Explain it in plain words
LinkedList.add(item)	Append at tail. No scan: O(1).
RiderList.add / DriverList.add	Reject duplicate ID/email/plate, find positions, insert into both ordered lists: O(n).
findById	Scan ID order; may stop once IDs pass the target. Worst case still O(n).
removeByName / city / type	Keep scanning after a match. Remove every match from both indexes: O(n).
RideList.findByRiderName	Check private rider or shared participants; add a ride only once even if two riders have the same name.

Practice questions

Why is get(i) O(n)? Because it starts at head. Why avoid get(i) in a full-list loop? The total could be 1+2+...+n = O(n^2). Why keep tail? It makes appending O(1). Why is sorted lookup not binary search? A linked list has no constant-time access to its middle.

B - Ibrahim: the system rules

Private scheduling: read it in this order

Open schedulePrivateRide. Explain validRide, the two findById calls, isAvailable for the rider and driver, creation of PrivateRide, rides.addRide, and the two history appends. If a check fails, return false before changing system state.

The overlap formula: know both comparisons

pickup1 < dropoff2 AND pickup2 < dropoff1. In code, compareTo(...) < 0 means earlier. Both comparisons must be true. The ranges are half-open, so touching endpoints are allowed.

Existing 09:00-10:00; new ride	Result and reason
09:30-10:30	Reject: they share time from 09:30 to 10:00.
08:00-11:00	Reject: the new ride surrounds the old ride.
09:00-10:00	Reject: exactly the same time.
10:00-11:00	Allow: one ends exactly when the other begins.
08:00-09:00	Allow: the other touching boundary.
10:00-09:00 or 09:00-09:00	Reject before checking histories: invalid duration.

Shared scheduling

Check the driver, then every rider. Reject missing riders, repeated IDs and any rider conflict. Require at least two distinct riders at creation. The local participant array keeps the validated objects. Only after validation create the ride, store it, and add it to every history.

Histories are essential

The same Ride object appears in the ride list and in the relevant histories. If you store the ride but forget the histories, later conflict checks may accept an overlapping ride. A failed scheduling attempt must not add anything to histories.

Complexity to say out loud

Private: find rider + find driver + scan two histories + insert ride = O(N+D+H+R). Shared: K rider lookups and history scans, duplicate/participant checks, and ride insertion = O(D+H+K*N+K*H+K^2+R). It is not just O(1) because the overlap formula has two comparisons.

B - Ibrahim: removals and CSV

Cascade removal: what else must change?

Removing a rider: delete their private rides; remove them from shared rides; keep a shared ride if someone remains; delete it if empty. Removing a driver: delete all rides assigned to that driver. In both cases clean every affected ride history so it cannot leave a false scheduling conflict.

Example: shared ride has riders 1 and 2, driver 10. Remove rider 1: rider 2 and driver 10 still have the ride. Remove rider 2: now delete the ride, and driver 10's history must lose it too.

Read these helpers

removeFromHistory scans one person's history and removes the ride ID. deleteRide removes storage and relevant histories. removeRider/removeDriver scan a copy of the ride list, so deleting stored rides does not disrupt traversal.

CSV load order and column order

Load riders, then drivers, then rides. A ride cannot point to a person who has not been loaded. Rider CSV order differs from the Rider constructor: CSV has email before phone; the constructor takes phone before email. loadRow rearranges those arguments.

Record	Example
Rider	1,Ahmed Ali,ahmed@example.com,0551234567,Riyadh
Driver	10,Fahad Salem,0551111111,ABC1234,SEDAN
Ride	SHARED,Mall,10/24/2026 11:00,10/24/2026 12:00,University,20,2;3

splitCSV handles commas inside quoted text; shared rider IDs use semicolons. parseDateTime converts the fixed text format into DateTime. CSV rides call the same schedule methods as Main, so file loading cannot bypass conflict checks.

What does false from a loader mean?

It means the file could not be read or at least one row failed. Valid rows are kept, and later rows are still attempted. Do not say it rolls back the entire file. Main displays the loader result; the system itself does not print.

Practice questions

Why remove histories too? To prevent stale conflicts. Why a snapshot? To traverse safely while deleting stored rides. Why check all shared riders first? To avoid partial updates. Can two drivers share one plate? No. Can two riders share a name? Yes, so name searches can return several matches.

C - Mohammed: models and menu

Inheritance: common behavior in one place

Person is abstract because it represents common person information, not a complete rider or driver. Rider and Driver extend Person. They share ID, name, phone and history, then add email/city or plate/type. Ride is also abstract; PrivateRide and SharedRide provide the actual rider participation.

Interfaces: the agreed contract

An interface states which operations a class must provide. For example, RideSharingSystem implements IRideSharingSystem. Calling through the interface lets Main and other classes use behavior without depending on private field layout. An abstract class can also provide fields and shared implementations.

Comparable and immutable values

Rider and Driver compare IDs. Ride compares pickup locations. DateTime compares date components in order. final fields stop IDs/time components from being reassigned after construction. compareTo returns negative, zero, or positive; zero means equal for the compared key, not that every field matches.

Menu responsibilities

Main first loads all three CSV files, then shows options 1-12 and 0. Each search/list option calls the system and displays every returned element's toString, or No results. Add options construct records/call scheduling and print success or failure. Only Main reads or prints console input/output.

Options	System calls
1 / 2 / 3 / 4	getAllRiders / searchRidersByHomeCity / searchRidersByName / addRider
5 / 6 / 7	getAllDrivers / searchDriversByVehicleType / addDriver
8 / 9 / 10 / 11	getAllRidesAlphabetically / searchRidesByPickupLocation / searchRidesByRiderName / getSharedRideParticipants
12 / 0	schedulePrivateRide or scheduleSharedRide / exit

Practice questions

Why not print inside search methods? Returning data allows direct testing and reuse. Why an enum? Only supported vehicle categories are allowed. Why catch invalid input? The menu should explain a mistake and continue. Why no removal menu option? The assignment specifies this exact menu; removal is available through system methods and tests.

What everyone must understand

Professor may ask	A clear answer
What is the project's main data structure?	Our own singly linked list, made of nodes with data and next references. We did not use Java collection classes.
What is stored in a ride history?	References to Ride objects associated with that person. Scheduling and deletion must keep them consistent.
What is the difference between private and shared?	Private has exactly one rider. Shared starts with at least two distinct riders and uses a participant list.
Why keep lists sorted during insertion?	To return ordered results in one linear traversal; we do not sort every time the user lists them.
What is worst-case O(n)?	The time can grow linearly with the number of records, for example when a lookup must inspect every node.
Can name removal stop after one match?	No. Names/cities/types are not unique; all matches must be removed.
What does compareTo < 0 mean?	This object comes before the other according to its comparison key.
What happens on invalid scheduling?	Return false and leave stored rides and histories unchanged.
What happens if two shared rides have one pickup location?	The participants query merges their participants and removes duplicate rider IDs.
Are list copies deep copies?	No. The list nodes are copied, but model objects are shared. Do not bypass system rules by externally editing those objects.

Five-minute team rehearsal

1. Each person explains their role in one minute.
2. Osamah draws three nodes and inserts/removes the middle one.
3. Ibrahim predicts a conflicting ride and a back-to-back ride.
4. Mohammed traces menu option 12 into the system call.
5. Everyone explains why deleting a driver also changes rider histories.

If you cannot explain a method, open it and trace a small example. Describe what changes and what stays consistent. Do not memorize a complexity claim without counting its scans.

Rehearse the actual demonstration

Start with the labelled sample data

Run run.bat --demo from the project folder. Main should report all three sample files loaded. The sample contains 5 riders, 4 drivers and 4 rides. These examples are synthetic; they are not the official assignment CSV files.

Action	Input / expected result
List people and rides	Options 1, 5 and 8. People display by name; rides begin Airport, Mall, Mall, University.
Show duplicate-name results	Option 3: Ahmed Ali. Two riders are returned; a first-name-only search Ahmed has no matches.
Show shared participants	Option 11: Mall. Riders 2, 3, 1 and 5 appear across two matching shared rides.
Show rider conflict	Option 12, PRIVATE, pickup Library, pickup 10/24/2026 09:30, dropoff 10/24/2026 10:30, destination Home, driver 30, rider 1. Rejected: rider 1 is already riding 09:00-10:00.
Show allowed touching time	Option 12, PRIVATE, Library, 10/24/2026 10:00, 10/24/2026 11:00, Home, driver 10, rider 1. Accepted: the previous ride ends at 10:00.
Show driver conflict	After the accepted ride, try 10:30-11:30 with driver 10 and rider 4. Rejected: driver 10 is busy.
Show a shared addition	Option 12, SHARED, Library, 10/24/2026 16:00, 10/24/2026 17:00, Home, driver 40, riders 1;2. Accepted.
Show validation	Try an invalid date or short phone when prompted. Main prints an invalid-input message and returns to the menu.
Show removals if asked	Open ProjectTest.schedulingTests and the cascade methods. The required menu has no removal option. Run test.bat to exercise these paths.

Before you submit

Add the official riders_100.csv, drivers_30.csv and rides_40.csv, then run the ordinary startup and check all load messages. They were not supplied with the downloaded repository. Verify the report cover. Review the code together and follow any course rules on permitted assistance. Submit the source ZIP and report as requested by the LMS.

Use this complete project instead of the earlier Role B package. It contains real A implementations, B logic and C models/menu/report. Do not upload temporary-stubs or the old Datetime.java spelling alongside DateTime.java.